OBSIDIA - DOMAIN STACK ONLY

Kernel Ragnarok 3001 + API 8000 + Bank / Trading / GPS

Support de lancement PowerShell - blocs colorees pour lecture rapide

Scope actuel : pas Graphiti, pas UI, pas Brody terminal, pas Neo4j dans cette stack. Neo4j peut rester connecte sur 127.0.0.1:7688, mais il est hors chemin critique pour les domaines.

1. Matrice cible

Bloc	Port / URL	Role	Statut attendu
Kernel Ragnarok	http://127.0.0.1:3001	Autorite decisionnelle domaine	BANK / TRADING / GPS visibles dans le terminal kernel
API Obsidia	http://127.0.0.1:8000	Bridge / transport / normalisation	Expose /api/live/kernel/adapters/*
Bank connector	connectors/bank_normal_flow.py	Injecte flux bank vers API puis kernel	decision_bank_... cree
Trading connector	connectors/trading_live.py	Injecte flux trading vers API puis kernel	decision_trading_... cree
GPS / Aviation connector	connectors/aviation_robo.py	Injecte flux GPS/Aviation vers API puis kernel	decision_gps_defense_aviation_... cree

Regle : l API 8000 ne doit pas decider seule. Elle doit transporter vers le kernel 3001 et retourner une decision kernel_decision.

2. Legende couleur des commandes

Variables	Chemins/URLs	Commandes	Ports	Stop/Danger	Commentaires
$REPO / $API	C:\... / http://...	python / node / Invoke	3001 / 8000	Stop-Process	# explication

Flux cible : Bank / Trading / GPS -> API 8000 -> Kernel Ragnarok 3001 -> fichiers decision_*.json. Rien d autre n est requis a ce palier.

BANK	TRADING	GPS	API 8000	KERNEL 3001

3. Bloc complet - domaine/kernel/API en une fois

A coller dans un PowerShell. Le script ouvre plusieurs fenetres : kernel, API, Bank, Trading, GPS. Il ne lance pas Neo4j, Graphiti, UI ou terminal Brody.

```powershell
# ============================================================
# OBSIDIA DOMAIN STACK ONLY
# Kernel Ragnarok 3001 + API 8000 + Bank/Trading/GPS
# PAS Graphiti / PAS Neo4j / PAS UI / PAS Brody terminal
# ============================================================

$ROOT = "C:\Users\User\Desktop\obsidia-engine-proof-core"
$REPO = Join-Path $ROOT "obsidia-x108-proofs_REMOTE_A5F21C6B"
$RT = Join-Path $REPO "runtime_terrain_bank_trading_gps"
$API = "http://127.0.0.1:8000"
$KERNEL_URL = "http://127.0.0.1:3001/kernel/ragnarok"

Write-Host "`n=== STOP DOMAIN STACK ONLY ===" -ForegroundColor Cyan

# Stop uniquement kernel / API / connecteurs domaines
Get-CimInstance Win32_Process |
  Where-Object {
    $_.CommandLine -match "server.kernel.sealed.cjs" -or
    $_.CommandLine -match "apps.obsidia_api.main:app" -or
    $_.CommandLine -match "connectors\\bank_normal_flow.py" -or
    $_.CommandLine -match "connectors\\trading_live.py" -or
    $_.CommandLine -match "connectors\\aviation_robo.py"
  } |
  ForEach-Object {
    Write-Host "Stopping PID=$($_.ProcessId) CMD=$($_.CommandLine)" -ForegroundColor Yellow
    Stop-Process -Id $_.ProcessId -Force -ErrorAction SilentlyContinue
  }

# Libere seulement les ports domaine/kernel/API
foreach ($port in @(3001, 8000)) {
  $pids = netstat -ano |
    Select-String ":$port\s" |
    Where-Object { $_.Line -match "LISTENING" } |
```

```
    ForEach-Object { ($_ -split "\s+")[-1] } |
    Sort-Object -Unique

  foreach ($pid in $pids) {
    if ($pid -and $pid -ne "0") {
      Write-Host "Stopping port $port PID=$pid" -ForegroundColor Yellow
      Stop-Process -Id ([int]$pid) -Force -ErrorAction SilentlyContinue
    }
  }
}

Start-Sleep -Seconds 2

Write-Host "`n=== START KERNEL RAGNAROK 3001 ===" -ForegroundColor Cyan

Start-Process powershell -ArgumentList @(
  "-NoExit",
  "-Command",
  "cd '$RT'; [Console]::Title='RAGNAROK KERNEL 3001 - DOMAIN AUTHORITY'; node .\server.kernel.sealed.cjs"
)

Start-Sleep -Seconds 5

Write-Host "`n=== START OBSIDIA API 8000 ===" -ForegroundColor Cyan

Start-Process powershell -ArgumentList @(
  "-NoExit",
  "-Command",
  "cd '$REPO'; [Console]::Title='OBSIDIA API 8000 - LIVE KERNEL BRIDGE'; `$env:PYTHONPATH='$REPO'; `$env:OBSIDIA_KERNEL_URL='$KERNEL_URL'; python -m uvicorn apps.obsidia_api.main:app --host 127.0.0.1 --port 8000"
)

Start-Sleep -Seconds 8

Write-Host "`n=== CHECK API + LIVE ROUTES ===" -ForegroundColor Cyan

try {
  Invoke-RestMethod "$API/" -TimeoutSec 10 | ConvertTo-Json -Depth 8
} catch {
  Write-Host "API 8000 NOT READY" -ForegroundColor Red
```

```
}

try {
  $openapi = Invoke-RestMethod "$API/openapi.json" -TimeoutSec 10
  $openapi.paths.PSObject.Properties |
    Where-Object { $_.Name -match "/api/live/kernel/adapters" } |
    Select-Object Name |
    Format-Table -AutoSize
} catch {
  Write-Host "OPENAPI ROUTES CHECK FAILED" -ForegroundColor Red
}

Write-Host "`n=== START DOMAIN CONNECTORS ===" -ForegroundColor Cyan

Start-Process powershell -ArgumentList @(
  "-NoExit",
  "-Command",
  "cd '$REPO'; [Console]::Title='BANK LIVE -> KERNEL'; `$env:PYTHONPATH='$REPO'; `$env:OBSIDIA_API_BASE='$API'; python .\connectors\bank_normal_flow.py"
)

Start-Process powershell -ArgumentList @(
  "-NoExit",
  "-Command",
  "cd '$REPO'; [Console]::Title='TRADING LIVE -> KERNEL'; `$env:PYTHONPATH='$REPO'; `$env:OBSIDIA_API_BASE='$API'; python .\connectors\trading_live.py"
)

Start-Process powershell -ArgumentList @(
  "-NoExit",
  "-Command",
  "cd '$REPO'; [Console]::Title='GPS AVIATION LIVE -> KERNEL'; `$env:PYTHONPATH='$REPO'; `$env:OBSIDIA_API_BASE='$API'; python .\connectors\aviation_robo.py"
)

Start-Sleep -Seconds 5

Write-Host "`n=== FINAL DOMAIN STACK CHECK ===" -ForegroundColor Cyan

netstat -ano | findstr ":3001"
netstat -ano | findstr ":8000"
```

```powershell
Get-CimInstance Win32_Process |
  Where-Object {
    $_.CommandLine -match "server.kernel.sealed.cjs" -or
    $_.CommandLine -match "apps.obsidia_api.main:app" -or
    $_.CommandLine -match "connectors\\bank_normal_flow.py" -or
    $_.CommandLine -match "connectors\\trading_live.py" -or
    $_.CommandLine -match "connectors\\aviation_robo.py"
  } |
  Select-Object ProcessId, Name, CommandLine |
  Format-Table -AutoSize

Write-Host "`nDOMAIN STACK READY" -ForegroundColor Green
Write-Host "Kernel : http://127.0.0.1:3001"
Write-Host "API    : http://127.0.0.1:8000"
Write-Host "Watch terminal RAGNAROK KERNEL 3001 for BANK / TRADING / GPS." -ForegroundColor Green
```

4. Commandes separees par instance

4.1 Stop domaine/kernel/API uniquement

```
cd "C:\Users\User\Desktop\obsidia-engine-proof-core\obsidia-x108-proofs_REMOTE_A5F21C6B"

Get-CimInstance Win32_Process |
  Where-Object {
    $_.CommandLine -match "server.kernel.sealed.cjs" -or
    $_.CommandLine -match "apps.obsidia_api.main:app" -or
    $_.CommandLine -match "connectors\\bank_normal_flow.py" -or
    $_.CommandLine -match "connectors\\trading_live.py" -or
    $_.CommandLine -match "connectors\\aviation_robo.py"
  } |
  ForEach-Object {
    Stop-Process -Id $_.ProcessId -Force -ErrorAction SilentlyContinue
  }

foreach ($port in @(3001, 8000)) {
  netstat -ano |
    Select-String ":$port\s" |
    Where-Object { $_.Line -match "LISTENING" } |
    ForEach-Object { Stop-Process -Id ([int](($_ -split "\s+")[-1])) -Force -ErrorAction SilentlyContinue }
}
```

4.2 Kernel Ragnarok - port 3001

```
cd "C:\Users\User\Desktop\obsidia-engine-proof-core\obsidia-x108-proofs_REMOTE_A5F21C6B\runtime_terrain_bank_trading_gps"
node .\server.kernel.sealed.cjs
```

4.3 API Obsidia / Live Kernel Bridge - port 8000

```
cd "C:\Users\User\Desktop\obsidia-engine-proof-core\obsidia-x108-proofs_REMOTE_A5F21C6B"

$env:PYTHONPATH = "C:\Users\User\Desktop\obsidia-engine-proof-core\obsidia-x108-proofs_REMOTE_A5F21C6B"
$env:OBSIDIA_KERNEL_URL = "http://127.0.0.1:3001/kernel/ragnarok"
```

```
python -m uvicorn apps.obsidia_api.main:app --host 127.0.0.1 --port 8000
```

4.4 Bank connector

```
cd "C:\Users\User\Desktop\obsidia-engine-proof-core\obsidia-x108-proofs_REMOTE_A5F21C6B"

$env:PYTHONPATH = "C:\Users\User\Desktop\obsidia-engine-proof-core\obsidia-x108-proofs_REMOTE_A5F21C6B"
$env:OBSIDIA_API_BASE = "http://127.0.0.1:8000"

python .\connectors\bank_normal_flow.py
```

4.5 Trading connector

```
cd "C:\Users\User\Desktop\obsidia-engine-proof-core\obsidia-x108-proofs_REMOTE_A5F21C6B"

$env:PYTHONPATH = "C:\Users\User\Desktop\obsidia-engine-proof-core\obsidia-x108-proofs_REMOTE_A5F21C6B"
$env:OBSIDIA_API_BASE = "http://127.0.0.1:8000"

python .\connectors\trading_live.py
```

4.6 GPS / Aviation connector

```
cd "C:\Users\User\Desktop\obsidia-engine-proof-core\obsidia-x108-proofs_REMOTE_A5F21C6B"

$env:PYTHONPATH = "C:\Users\User\Desktop\obsidia-engine-proof-core\obsidia-x108-proofs_REMOTE_A5F21C6B"
$env:OBSIDIA_API_BASE = "http://127.0.0.1:8000"

python .\connectors\aviation_robo.py
```

4.7 Check routes API live kernel

```
$openapi = Invoke-RestMethod "http://127.0.0.1:8000/openapi.json"

$openapi.paths.PSObject.Properties |
  Where-Object { $_.Name -match "/api/live/kernel/adapters" } |
```

```
  Select-Object Name |
  Format-Table -AutoSize
```

4.8 Check ports

```
netstat -ano | findstr ":3001"
netstat -ano | findstr ":8000"
```

4.9 Logs attendus cote kernel

```
[BRIDGE] Routing -> Domain: bank
[KERNEL_DECISION]
[SAVE] decision_bank_...

[BRIDGE] Routing -> Domain: trading
[KERNEL_DECISION]
[SAVE] decision_trading_...

[BRIDGE] Routing -> Domain: gps_defense_aviation
[KERNEL_DECISION]
[SAVE] decision_gps_defense_aviation_...
```

5. Depannage rapide

> WinError 10048 sur 8000 = une ancienne API utilise deja le port. Tue uniquement le PID en LISTENING sur 8000, puis relance l API.

5.1 Liberer le port 8000 seulement

```
# Probleme: WinError 10048 sur 8000 = port deja occupe
$pids = netstat -ano |
  Select-String ":8000\s" |
  Where-Object { $_.Line -match "LISTENING" } |
  ForEach-Object { ($_ -split "\s+")[-1] } |
```

```
  Sort-Object -Unique

$pids | ForEach-Object {
  Write-Host "Killing port 8000 PID=$_" -ForegroundColor Yellow
  Stop-Process -Id ([int]$_) -Force -ErrorAction SilentlyContinue
}
```